

Node.js

Node.js란?

- React는 Node.js를 기반으로 작동함
- 자바스크립트
 - 웹 브라우저에 내장된 자바스크립트 엔진으로 실행됨
 - 자바스크립트를 실행하는 웹 브라우저를 자바스크립트의 구동 환경이라는 뜻에서 ‘자바스크립트 런타임’이라고도 표현
- Node.js
 - Chrome V8 JavaScript 엔진으로 빌드 된 JavaScript 런타임
 - 웹 페이지 내부에 필요한 아주 단순한 기능만을 개발하기 위해 만들어짐
 - 웹 브라우저외에 웹 서버나 모바일 어플리케이션을 개발할 때도 자바스크립트를 이용하여 실행할 수 있게 됨
 - 매우 유연하게 설계되고, 생산성이 높음
- React를 잘 다루기 위해서는 Node.js 학습이 선행되어야 함
- Node.js에는 리액트를 효율적으로 다루는 여러 도구들이 내장되어 있을 뿐만 아니라, 궁극적으로 Node.js가 리액트로 만든 자바스크립트 어플리케이션을 구동하기 때문임

Node.js 환경 설정하기

- JavaScript 실행 환경(Run Time) : 구동기
- Node.js 다운로드
 - <https://nodejs.org/ko>
 - 윈도우 사용자 계정 영문 확인
 - PC 계정 이름이 영문으로 되어 있어야 함

The screenshot shows the Node.js website homepage. The navigation bar at the top includes links for '학습' (Learn), '소개' (Intro), '다운로드' (Download), '블로그' (Blog), '문서' (Docs), 'Beta Docs', '기여하기' (Contribute), and '인증' (Auth). The '다운로드' link is highlighted with a red box. Below the navigation bar, there is a banner for 'Run JavaScript Everywhere' with the text: 'Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.' Below this banner are two buttons: 'Get Node.js®' and 'Get security support for EOL Node.js versions'. To the right of the banner is a code editor showing a JavaScript snippet for creating an HTTP server. At the bottom, there are buttons for 'v24.19.0 Latest LTS' and 'v26.7.0 Latest Release'. The footer contains copyright information and various policy links.

nodejs 학습 소개 **다운로드** 블로그 문서 Beta Docs 기여하기 인증

Be Heard Take the Node.js User Survey 2026

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support for EOL Node.js versions

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript </> 문법보도에 복사

Learn more what Node.js is able to offer with our Learning materials.

v24.19.0 Latest LTS v26.7.0 Latest Release

Copyright OpenJS Foundation and Node.js contributors. All rights reserved. The OpenJS Foundation has registered trademarks and uses trademarks. For a list of trademarks of the OpenJS Foundation, please see our Trademark Policy and Trademark List. Trademarks and logos not indicated on the list of OpenJS Foundation trademarks are trademarks™ or registered® trademarks of their respective holders. Use of them does not imply any affiliation with or endorsement by them.

OpenJS Foundation AI Coding Assistants Policy Bylaws 행동 강령 Cookie Policy 개인정보 처리방침 보안 정책 Terms of Use Trademark List 상표 정책

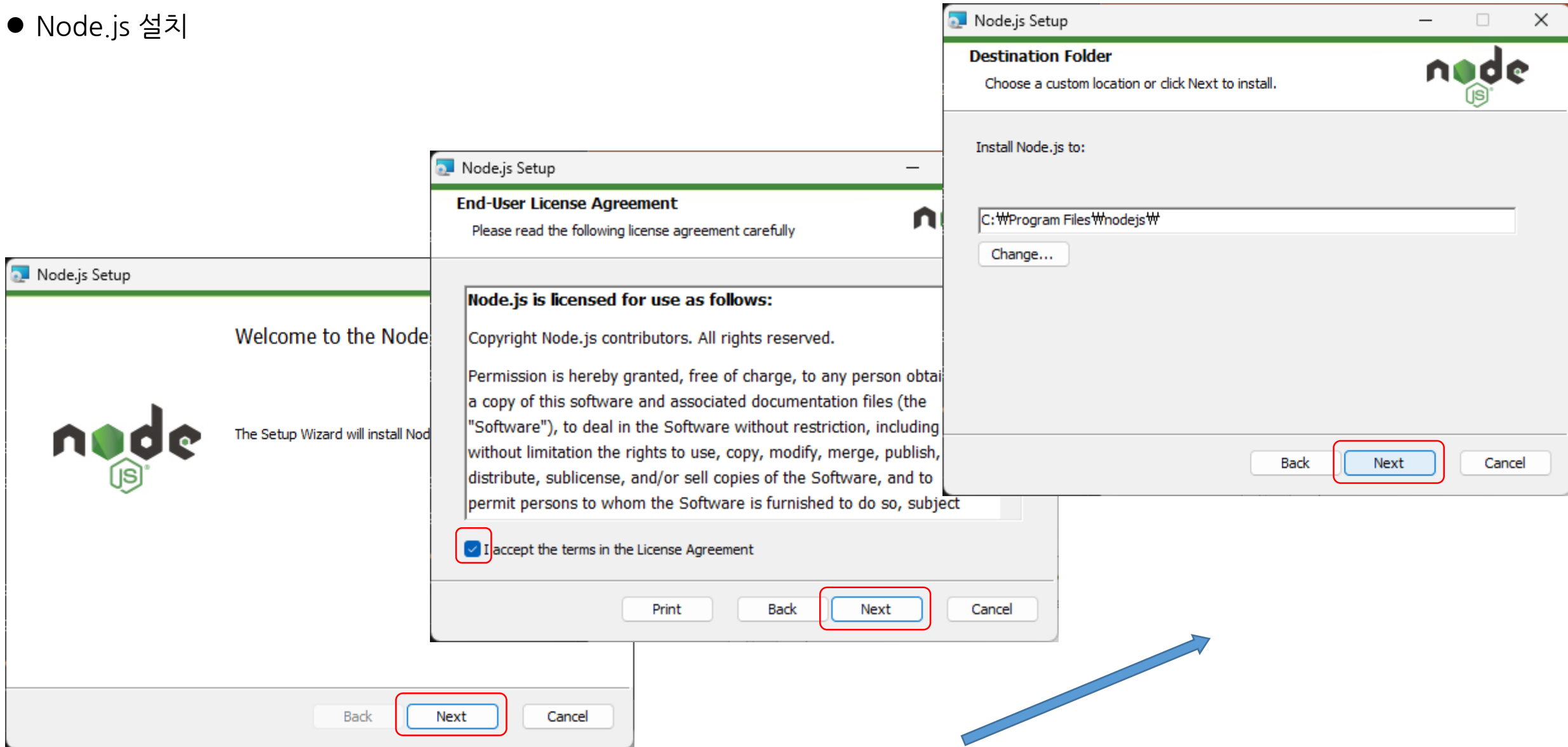
Node.js 환경 설정하기

- JavaScript 실행 환경(Run Time) : 구동기
- Node.js 다운로드
 - <https://nodejs.org/ko>
 - 윈도우 사용자 계정 영문 확인
 - PC 계정 이름이 영문으로 되어 있어야 함

The screenshot shows the Node.js download page in Korean. At the top, there's a navigation bar with links like '학습', '소개', '다운로드', '블로그', '문서', 'Beta Docs', '기여하기', and '인증'. A search bar is on the right. The main heading is 'Node.js® 다운로드'. Below it, there are dropdown menus for 'Node.js®' (v24.19.0 LTS), '플랫폼' (Windows), '환경에서' (Docker), '방식으로' (npm), and '플러그인' (사용해 설치하세요). A green banner below the dropdowns says '정보 Want new features sooner? Get the latest Node.js version instead and try the latest improvements!'. The main content area contains a code block with instructions for installing Node.js using Docker. Below the code block, there's a 'PowerShell' button and a link to '플랫폼보드에 복사'. A note below the code block says 'Docker는 컨테이너 플랫폼입니다. 문제가 발생하면 Docker's 웹사이트를 방문하세요.' Below this, there are two buttons: '또는 x64 아키텍처가 실행 중인 Windows 환경에서 미리 빌드된 Node.js®를 다운로드하세요.' and 'Windows 설치 프로그램 (.msi) Standalone Binary (.zip)'. The 'Windows 설치 프로그램 (.msi)' button is highlighted with a red box. At the bottom, there are several links for more information: '이 버전의 변경 내역', '이 버전의 블로그 게시물을 확인하세요.', '서명된 SHASUMS를 검증하는 방법을 배우세요.', '소스에서 Node.js를 빌드하는 방법을 확인하세요.', and 'nightly 바이너리나 모든 이전 릴리스 또는 다른 플랫폼용 비공식 바이너리를 확인하세요.'

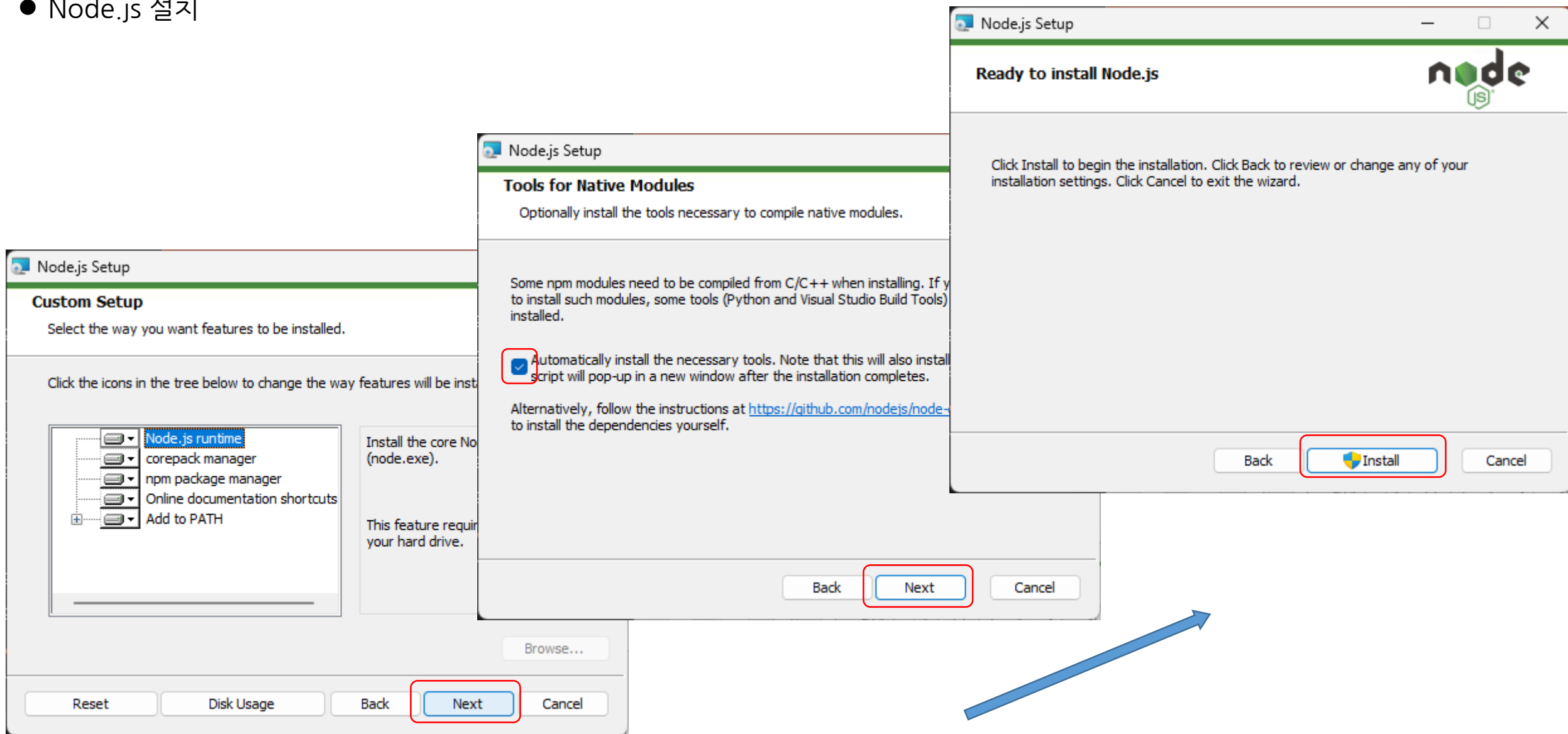
Node.js 환경 설정하기

- Node.js 설치



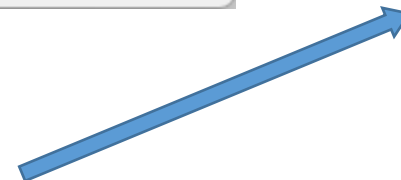
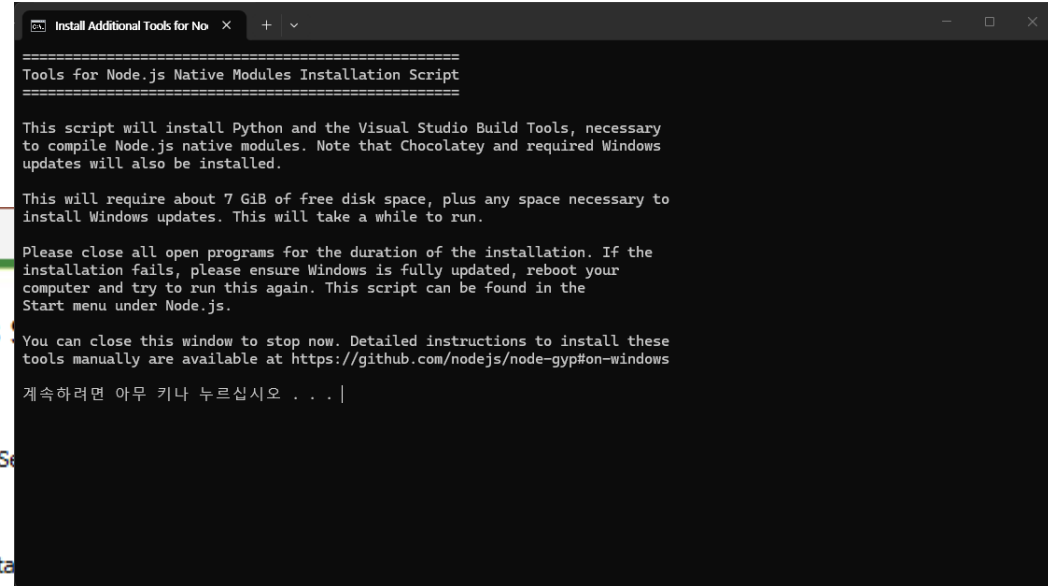
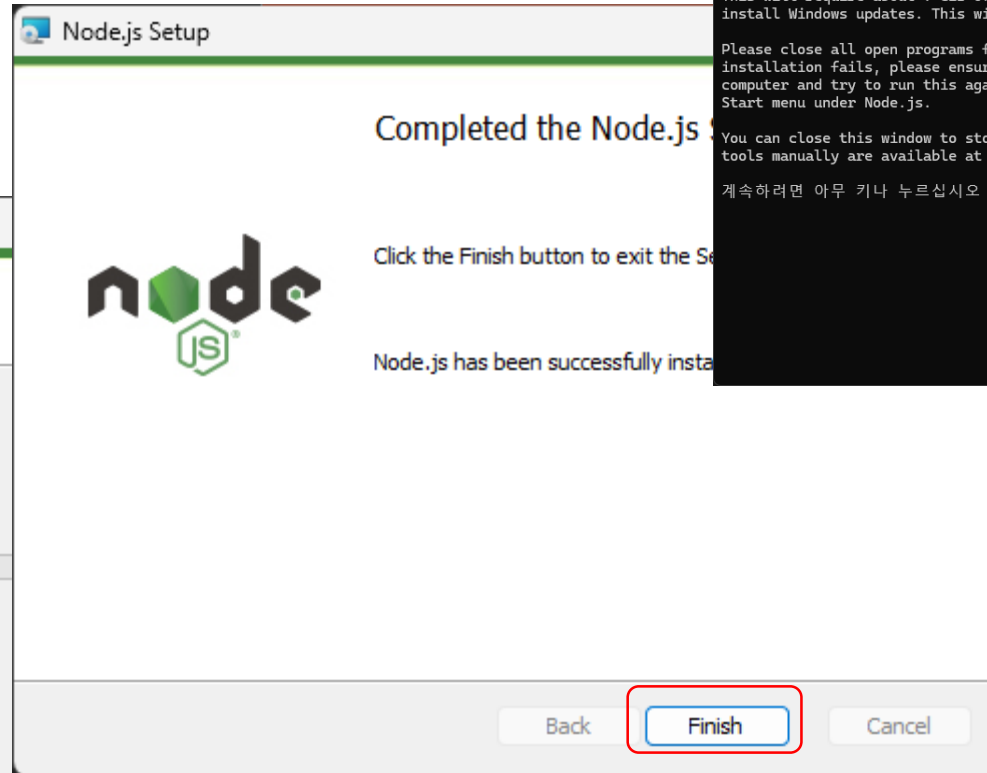
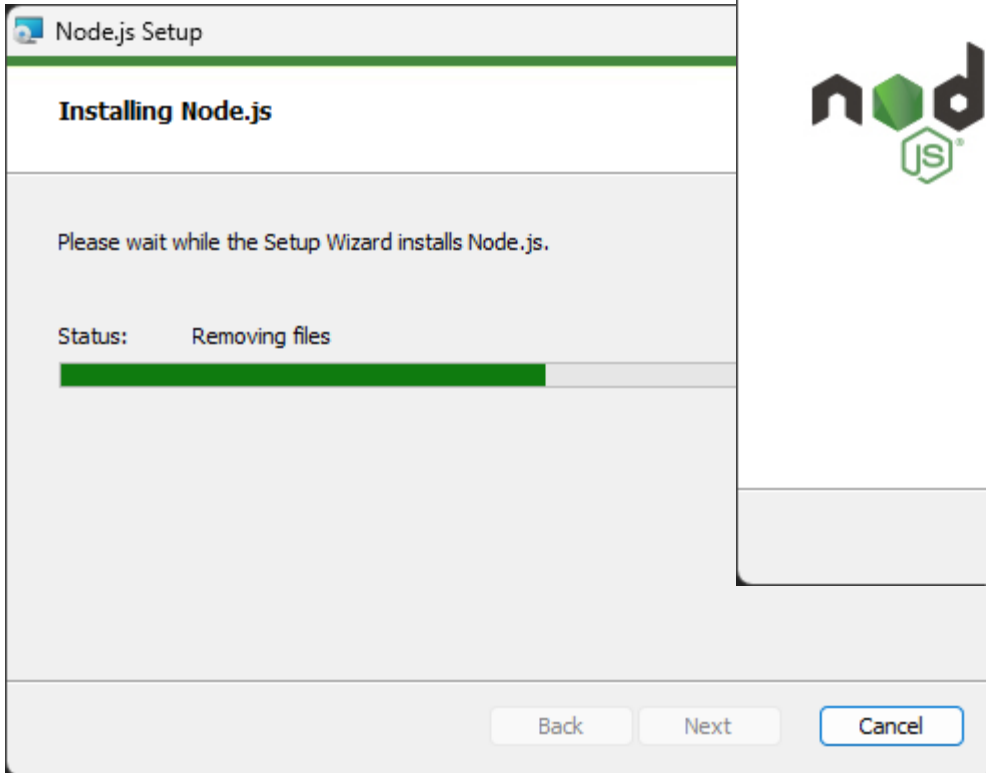
Node.js 환경 설정하기

- Node.js 설치



Node.js 환경 설정하기

- Node.js 설치



Node.js 환경 설정하기

- Node.js 설치

```
관리자: Windows PowerShell
경고: 'choco' was found at 'C:\ProgramData\chocolatey\bin\choco.exe'.
경고: An existing Chocolatey installation was detected. Installation will not continue. This script will not overwrite
existing installations.
If there is no Chocolatey installation at 'C:\ProgramData\chocolatey', delete the folder and attempt the installation
again.

Please use choco upgrade chocolatey to handle upgrades of Chocolatey itself.
If the existing installation is not functional or a prior installation did not complete, follow these steps:
- Backup the files at the path listed above so you can restore your previous installation if needed.
- Remove the existing installation manually.
- Rerun this installation script.
- Reinstall any packages previously installed, if needed (refer to the lib folder in the backup).

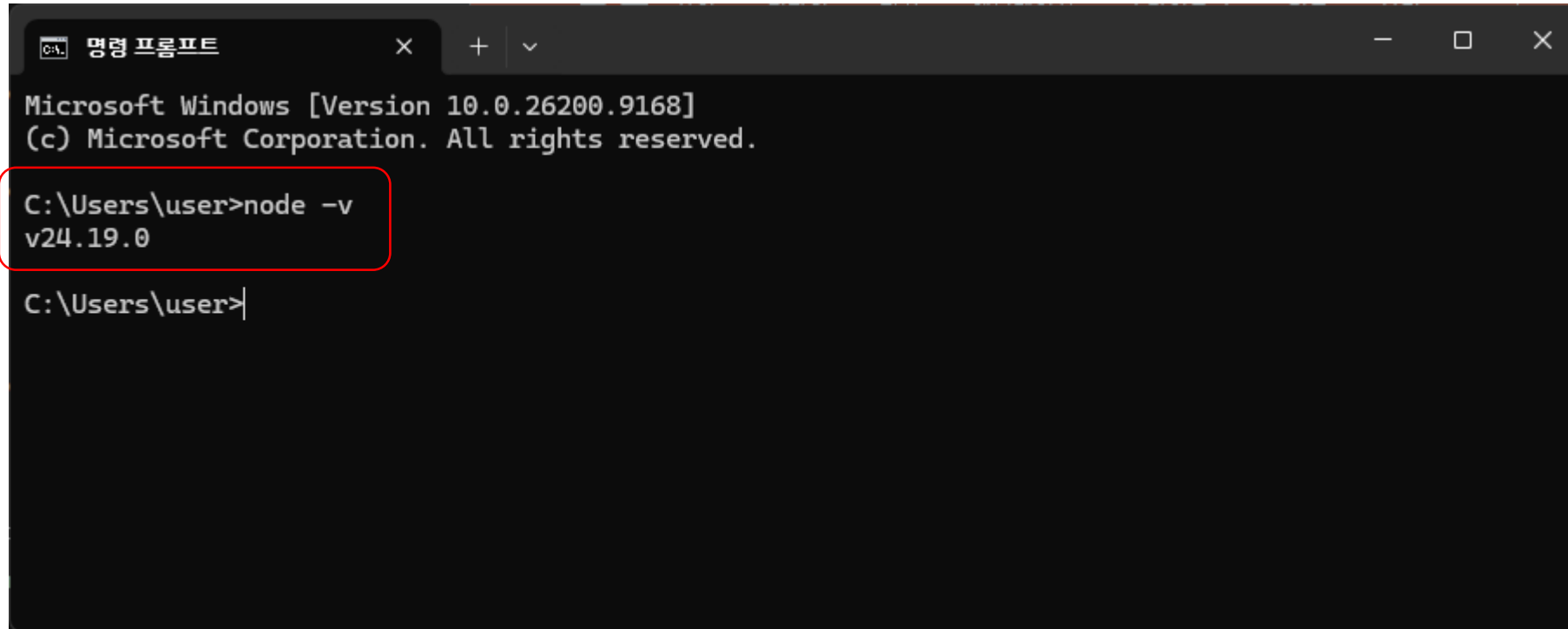
Once installation is completed, the backup folder is no longer needed and can be deleted.
Chocolatey v2.7.2
Upgrading the following packages:
python:visualstudio2025-workload-vctools
By upgrading, you accept licenses for the packages.
python is not installed. Installing...
Downloading package from source 'https://community.chocolatey.org/api/v2/'
Progress: Downloading chocolatey-compatibility.extension 1.0.0... 100%

chocolatey-compatibility.extension v1.0.0 [Approved]
chocolatey-compatibility.extension package files upgrade completed. Performing other installation steps.
Installed/updated chocolatey-compatibility extensions.
The upgrade of chocolatey-compatibility.extension was successful.
Deployed to 'C:\ProgramData\chocolatey\extensions\chocolatey-compatibility'
Downloading package from source 'https://community.chocolatey.org/api/v2/'
Progress: Downloading chocolatey-core.extension 1.4.0... 100%

chocolatey-core.extension v1.4.0 [Approved]
chocolatey-core.extension package files upgrade completed. Performing other installation steps.
Installed/updated chocolatey-core extensions.
The upgrade of chocolatey-core.extension was successful.
Deployed to 'C:\ProgramData\chocolatey\extensions\chocolatey-core'
```

Node.js 환경 설정하기

- Node.js 설치 확인하기



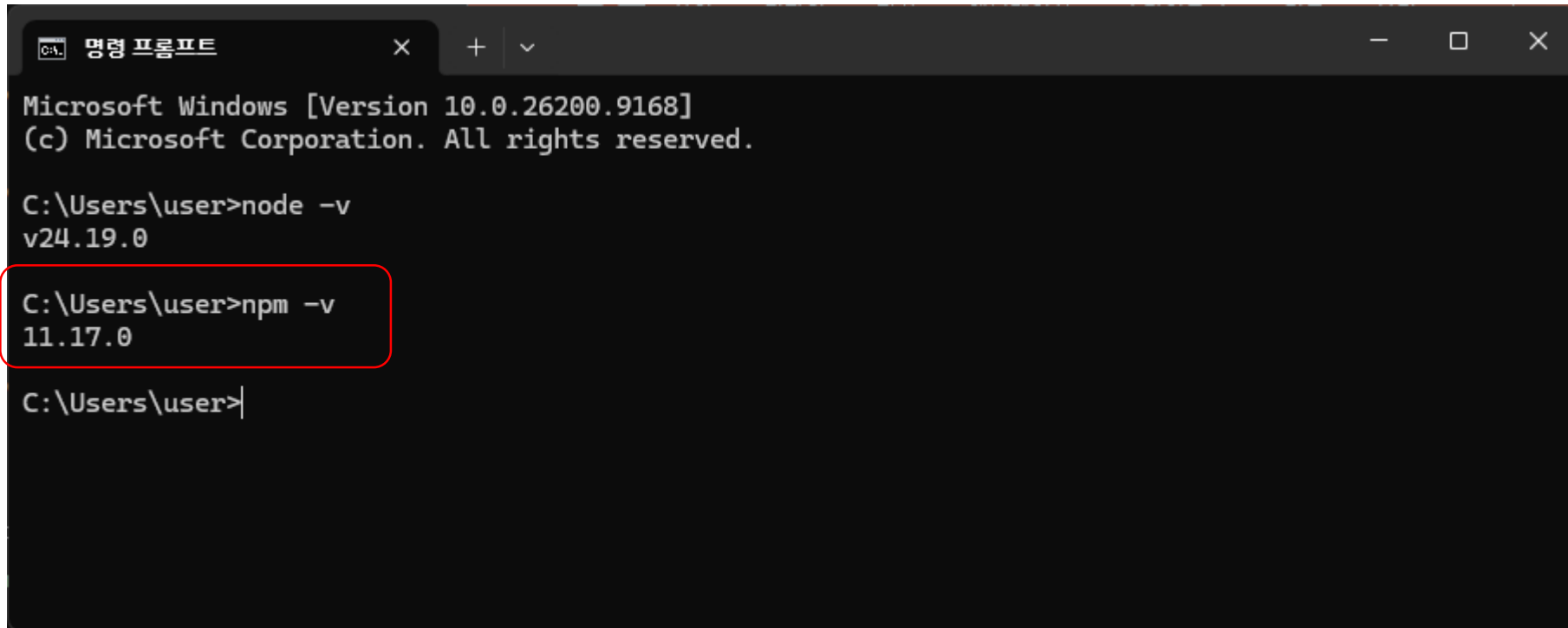
```
명령 프롬프트
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>node -v
v24.19.0

C:\Users\user>
```

Node.js 환경 설정하기

- npm 버전 확인하기



```
명령 프롬프트
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>node -v
v24.19.0

C:\Users\user>npm -v
11.17.0

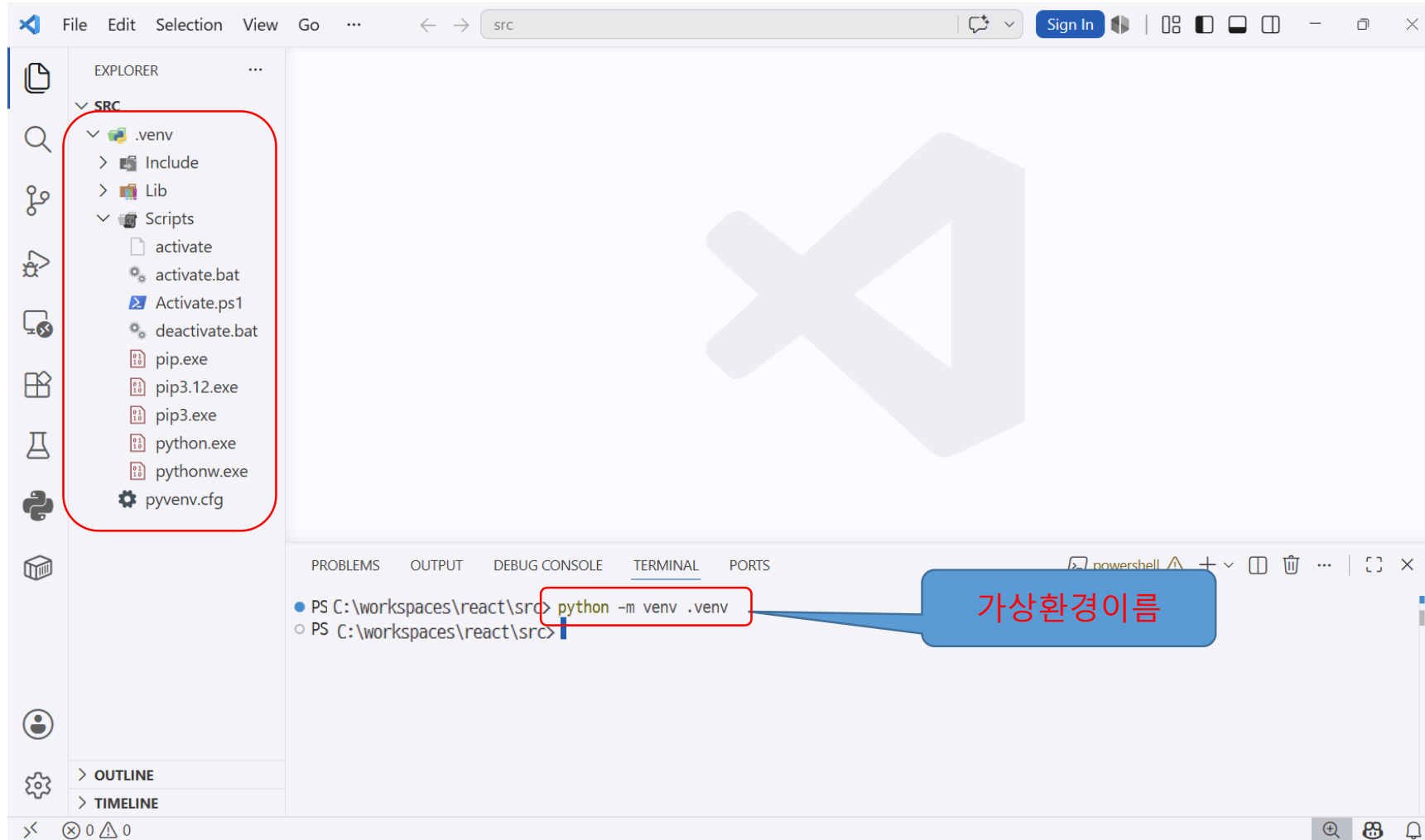
C:\Users\user>
```

The screenshot shows a Windows Command Prompt window with a dark background. The title bar reads '명령 프롬프트'. The window displays the output of two commands: 'node -v' which returns 'v24.19.0', and 'npm -v' which returns '11.17.0'. The 'npm -v' command and its output are highlighted with a red rounded rectangle.

Node.js 환경 설정하기

- (실습) 가상 환경 만들기

1. VS Code에서 프로젝트 폴더로 사용할 폴더를 열고 **Ctrl** + **`** 을 눌러 터미널 창 열기



Node.js 환경 설정하기

- (실습) 가상 환경 만들기

2. VS Code의 터미널 창에 다음과 같이 입력해 가상 환경 활성화



The screenshot shows the VS Code interface with the 'TERMINAL' tab selected. The terminal is running a PowerShell session. The first command executed is `python -m venv .venv`. The second command is `.\.venv\Scripts\activate`, which is highlighted with a red box. The prompt then changes to `(.venv) PS C:\workspaces\react\src>`, where `(.venv)` is also highlighted with a red box. The terminal title bar indicates it is a 'powershell' window.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell ! + v [ ] [ ] ... | [ ] X
```

```
● PS C:\workspaces\react\src> python -m venv .venv
```

```
● PS C:\workspaces\react\src> .\.venv\Scripts\activate
```

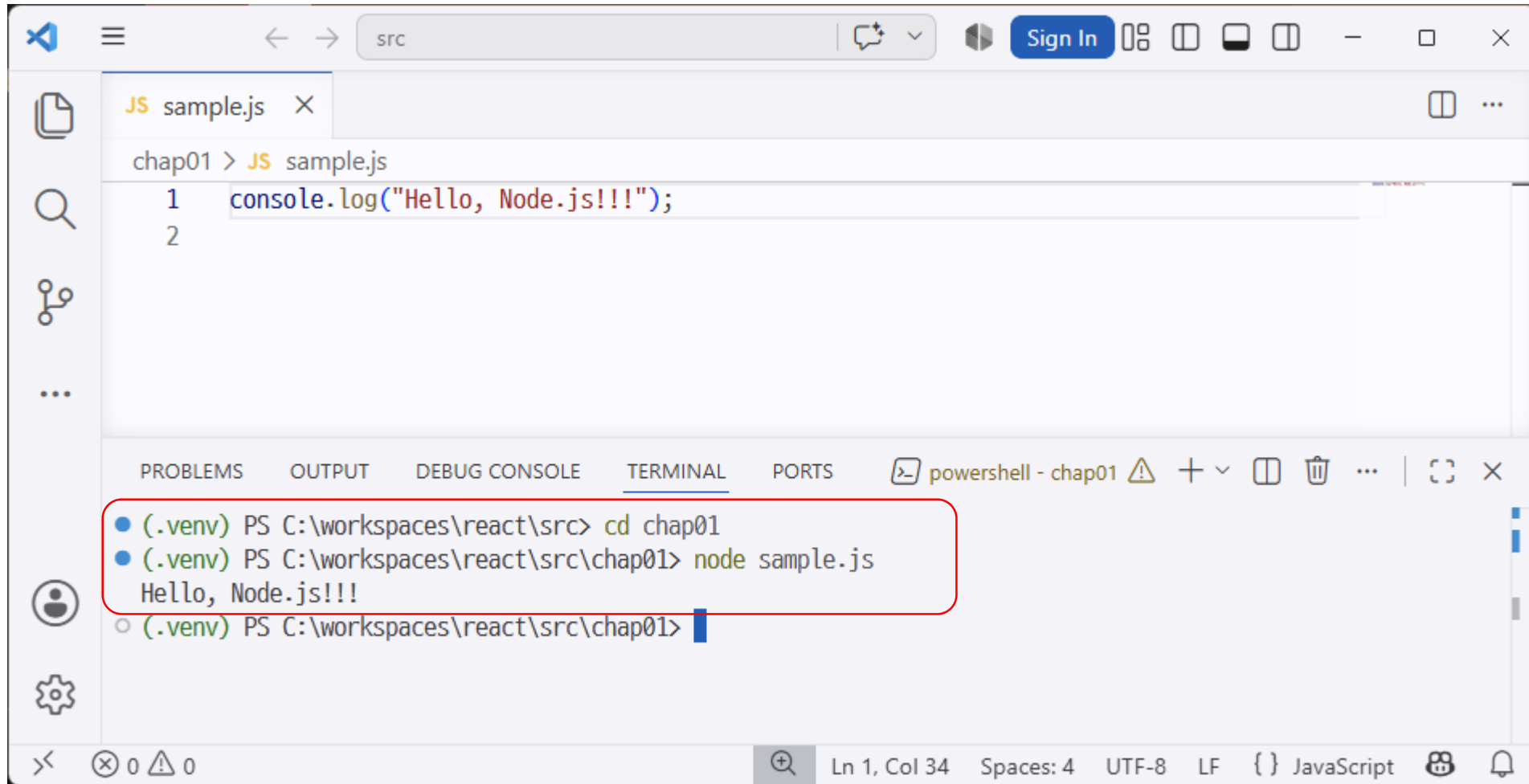
```
○ (.venv) PS C:\workspaces\react\src> |
```

3. VS Code를 실행하고 chap01 폴더 생성 후 sample.js 파일 생성

```
console.log("hello");
```

Node.js로 자바스크립트 실행하기

- Node.js로 sample.js 실행하기 - chap01 폴더로 이동 후 다음 명령어 실행
 - node sample.js



The screenshot shows the Visual Studio Code editor with a file named `sample.js` open. The file contains two lines of JavaScript code: `console.log("Hello, Node.js!!!");`. The editor's interface includes a sidebar on the left with icons for Explorer, Search, Source Control, and Run and Debug. The bottom panel shows the TERMINAL tab, which contains the following commands and output:

```
(.venv) PS C:\workspaces\react\src> cd chap01
(.venv) PS C:\workspaces\react\src\chap01> node sample.js
Hello, Node.js!!!
(.venv) PS C:\workspaces\react\src\chap01>
```

The first two lines of the terminal output are highlighted with a red rectangle. The status bar at the bottom indicates the current file is `sample.js` at line 1, column 34, with a space of 4, using UTF-8 encoding and LF line endings.

Node.js 패키지

- 하나의 프로젝트에서 여러 자바스크립트 파일을 Node.js를 이용해 실행할 때는 패키지(package) 형태로 구성
- 패키지는 Node.js에서 여러 개의 자바스크립트 파일을 실행하고 관리하는 일종의 관리 단위
- 패키지 만들기
 - 패키지를 생성하려면 npm(Node Package Manager)을 이용해야 함
 - npm이 Node.js 패키지를 관리하는 도구임
 - npm init 명령어를 vs code 터미널 창에서 입력하여 Node.js 패키지를 초기화함
 - 패키지 이름을 입력하라는 프롬프트가 나타나는데, chap01을 입력하고 enter 키를 누름
 - 이후 버전, 설명 등의 패키지 구성에 필요한 사항을 계속 물어오는데, 아무것도 입력하지 말고 enter 키를 누름
 - 마지막 Is This OK? 문구가 뜨면 yes를 입력하고 enter 누름
 - 다음 페이지 참조

Node.js 패키지

- npm init 실행 화면

```
(.venv) PS C:\workspaces\react\src\chap01> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json file.

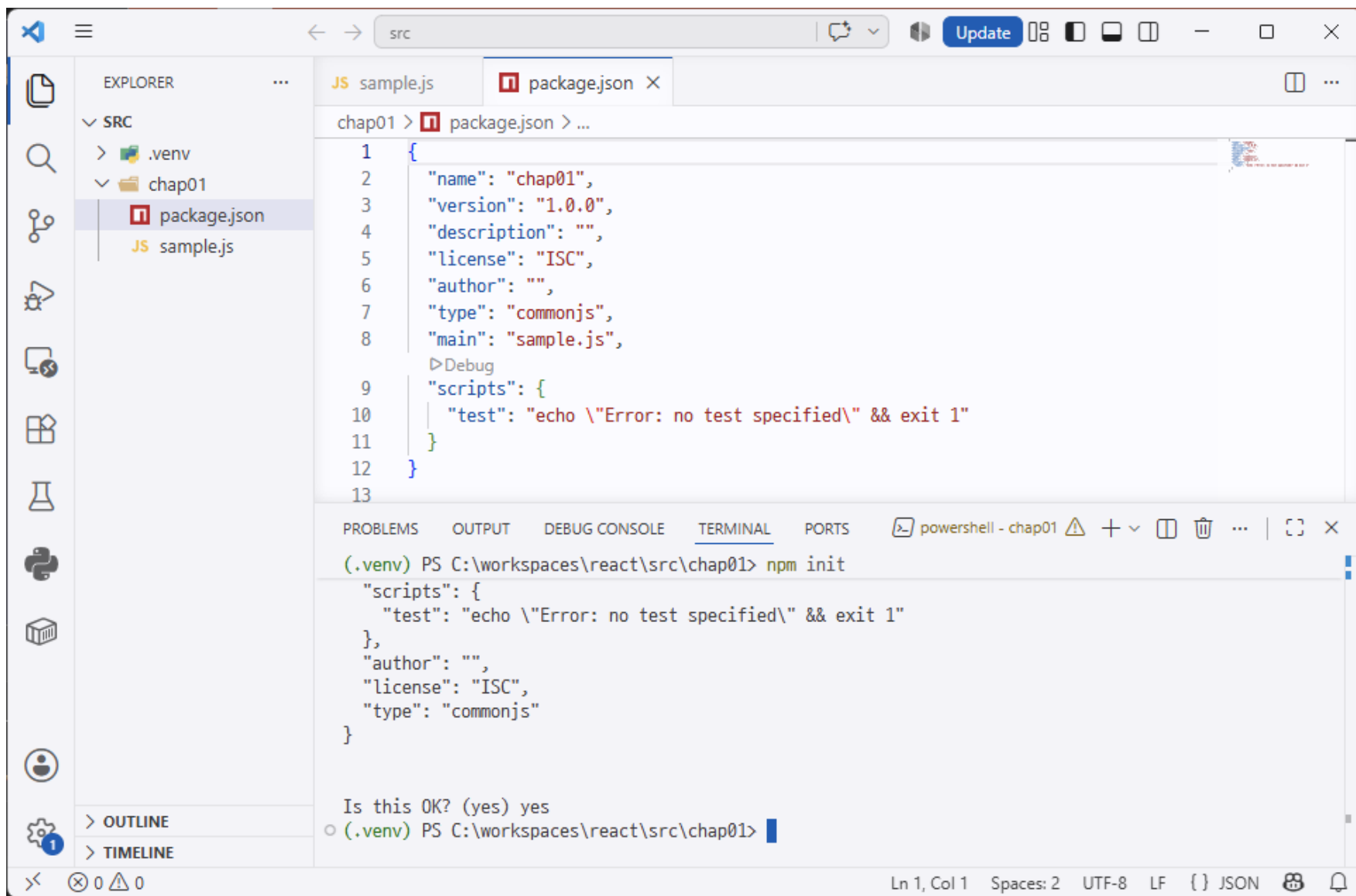
Press ^C at any time to quit.
package name: (chap01) chap01
version: (1.0.0)
description:
entry point: (sample.js)
test command:
git repository:
keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\workspaces\react\src\chap01\package.json:

{
  "name": "chap01",
  "version": "1.0.0",
  "description": "",
  "main": "sample.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

Is this OK? (yes) yes
(.venv) PS C:\workspaces\react\src\chap01>
```

Node.js 패키지

- package.json 파일 생성



```
chapg01 > package.json > ...
1 {
2   "name": "chapg01",
3   "version": "1.0.0",
4   "description": "",
5   "license": "ISC",
6   "author": "",
7   "type": "commonjs",
8   "main": "sample.js",
9   "scripts": {
10     "test": "echo \"Error: no test specified\" && exit 1"
11   }
12 }
13
```

```
(.venv) PS C:\workspaces\react\src\chapg01> npm init
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"author": "",
"license": "ISC",
"type": "commonjs"
}

Is this OK? (yes) yes
(.venv) PS C:\workspaces\react\src\chapg01>
```

Node.js 패키지

● 패키지 스크립트 사용하기

- Node.js 패키지의 package.json에는 scripts라는 항목이 있음
- 이 scripts 항목은 복잡한 명령어를 간단한 명령어로 변경하는 일종의 매크로 기능을 지원함
- scripts 항목을 직접 수정하고 사용해 보겠음
- 현재 패키지의 루트 폴더 아래에 index.js 파일 생성하고, console.log("index run"); 코드 추가
- package.json의 scripts 항목을 다음과 같이 수정



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure: SRC > .env > chap01 > index.js, package.json, and sample.js. The package.json file is selected. The main editor shows the content of package.json, with the 'scripts' property highlighted by a red rectangle. The code in package.json is as follows:

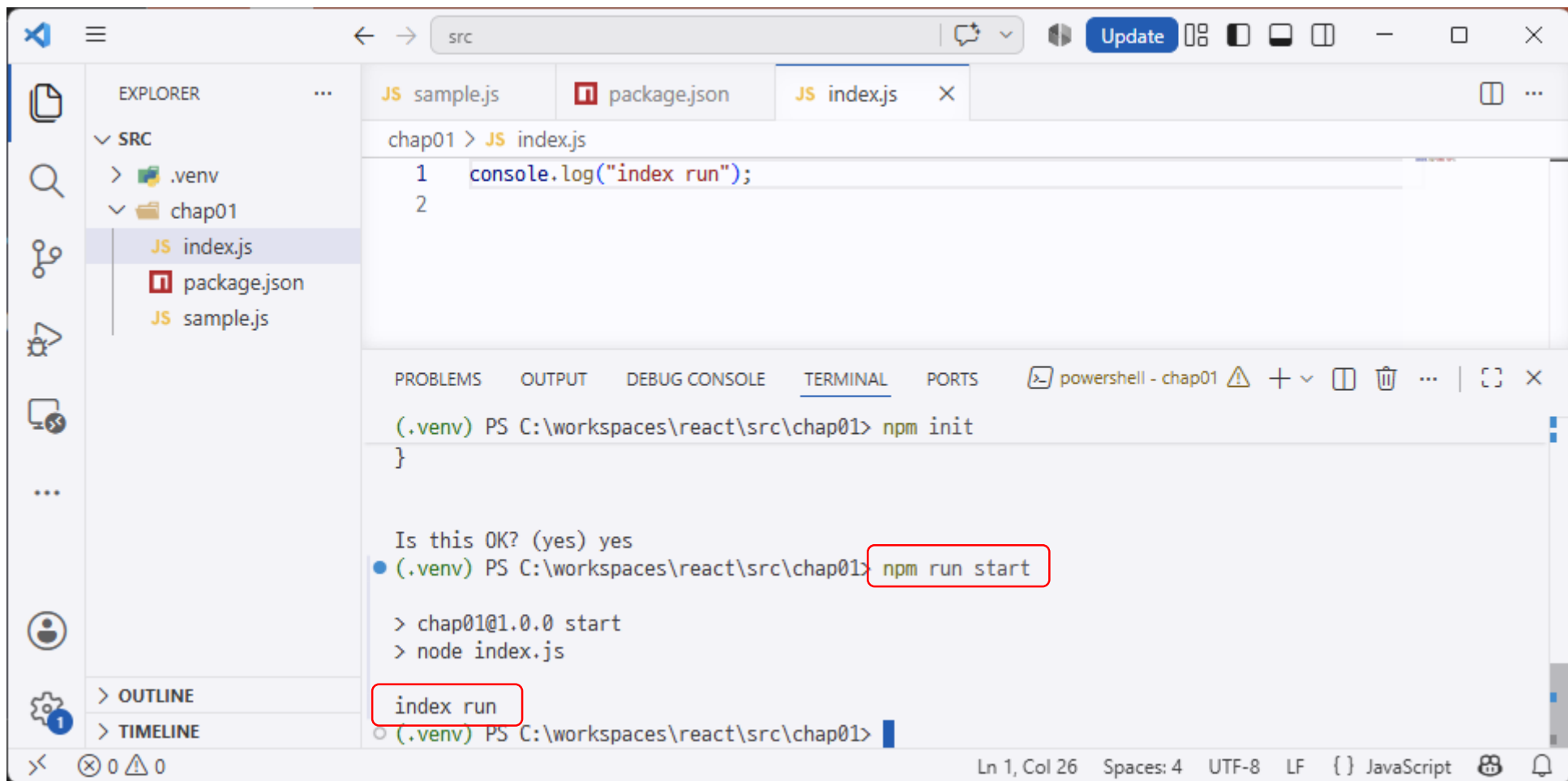
```
1 {
2   "name": "chap01",
3   "version": "1.0.0",
4   "description": "",
5   "license": "ISC",
6   "author": "",
7   "type": "commonjs",
8   "main": "sample.js",
9   "scripts": {
10     "start": "node index.js",
11     "test": "echo \\\"Error: no test specified\\\" && exit 1"
12   }
13 }
```

Node.js 패키지

- 패키지 스크립트 사용하기

- 다음 명령어를 터미널에서 입력하여 실행

- npm run start



```
src
├── .env
├── chap01
│   ├── index.js
│   ├── package.json
│   └── sample.js
└── ...
```

```
1 console.log("index run");
2
```

```
(.venv) PS C:\workspaces\react\src\chap01> npm init
}

Is this OK? (yes) yes
(.venv) PS C:\workspaces\react\src\chap01> npm run start

> chap01@1.0.0 start
> node index.js

index run
(.venv) PS C:\workspaces\react\src\chap01>
```

Node.js 모듈 시스템

- 모듈과 모듈 시스템

- 모듈

- 재사용할 수 있고, 독립적으로 존재하는 프로그램의 일부
 - 자바스크립트에서는 독립된 하나의 파일

- 모듈 시스템

- 모듈을 사용하는 방법

- ES 모듈 시스템

- ECMAScript 모듈 시스템의 약자. 줄여서 ESM이라고 함
 - ESM은 가장 최근에 개발된 모듈 시스템으로, 리액트, Vue와 같은 최신 프론트엔드 기술은 모두 ESM을 채택하고 있음

Node.js 모듈 시스템

- ES 모듈 시스템

- ESM 사용 설정하기

- Node.js는 기본적으로 ESM이 아닌 CJS 모듈 시스템을 사용함
 - 따라서 ESM 모듈 시스템을 사용하려면, package.json에서 설정을 변경해야 함
 - “type”의 값을 commonjs에서 module로 변경

```
{
  "name": "chap01",
  "version": "1.0.0",
  "description": "",
  "license": "ISC",
  "author": "",
  "type": "module",
  "main": "sample.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  }
}
```

Node.js 모듈 시스템

- ES 모듈 시스템

- 개별 내보내기

- circle.js 파일 생성

```
... ( 생략 ) ...
```

```
export { PI, getArea, getCircumference };
```

- 개별 불러오기

- index.js 에서 다음 코드 작성

```
// console.log("index run");
```

```
import { PI, getArea, getCircumference } from "./circle.js";
```

```
console.log(PI, getArea(1), getCircumference(1))
```

Node.js 모듈 시스템

- ES 모듈 시스템

- 터미널 창에서 실행하여 결과 확인

```
(.venv) PS C:\workspaces\react\src\chap01> npm run start
```

```
> chap01@1.0.0 start
```

```
> node index.js
```

```
3.141592 3.141592 6.283184
```

```
(.venv) PS C:\workspaces\react\src\chap01>
```

Node.js 모듈 시스템

- ES 모듈 시스템

- 전부 불러오기

- index.js 파일을 다음과 같이 수정

```
import * as circle from "./circle.js";  
  
console.log(circle.PI, circle.getArea(1), circle.getCircumference(1))
```

- 모듈 circle.js가 내보낸 값을 모두 불러와 변수 circle에 프로퍼티로 저장함
 - circle.PI와 같이 점 표기법으로 특정 모듈에 접근함
 - npm run start 명령으로 출력 결과 확인

Node.js 모듈 시스템

- ES 모듈 시스템

- 기본값으로 내보내기

- ESM에서는 export 키워드 다음에 default를 붙여 모듈의 기본값으로 내보낼 수 있음
 - export로 내보낸 값은 중괄호로 내보낸 이름과 동일한 이름으로 불러와야 함
 - 반면 export default 명령으로 내보내면 자유롭게 이름을 지정할 수 있음
 - circle.js를 다음과 같이 수정

```
... ( 생략 ) ...
```

```
export default { PI, getArea, getCircumference };
```

- PI, getArea, getCircumference를 프로퍼티로 하는 객체를 기본값으로 내보냄
 - circle.js에서 기본값으로 내보낸 값들을 index.js에서 불러옴

```
import circle from "./circle.js";
```

```
console.log(circle.PI, circle.getArea(1), circle.getCircumference(1))
```

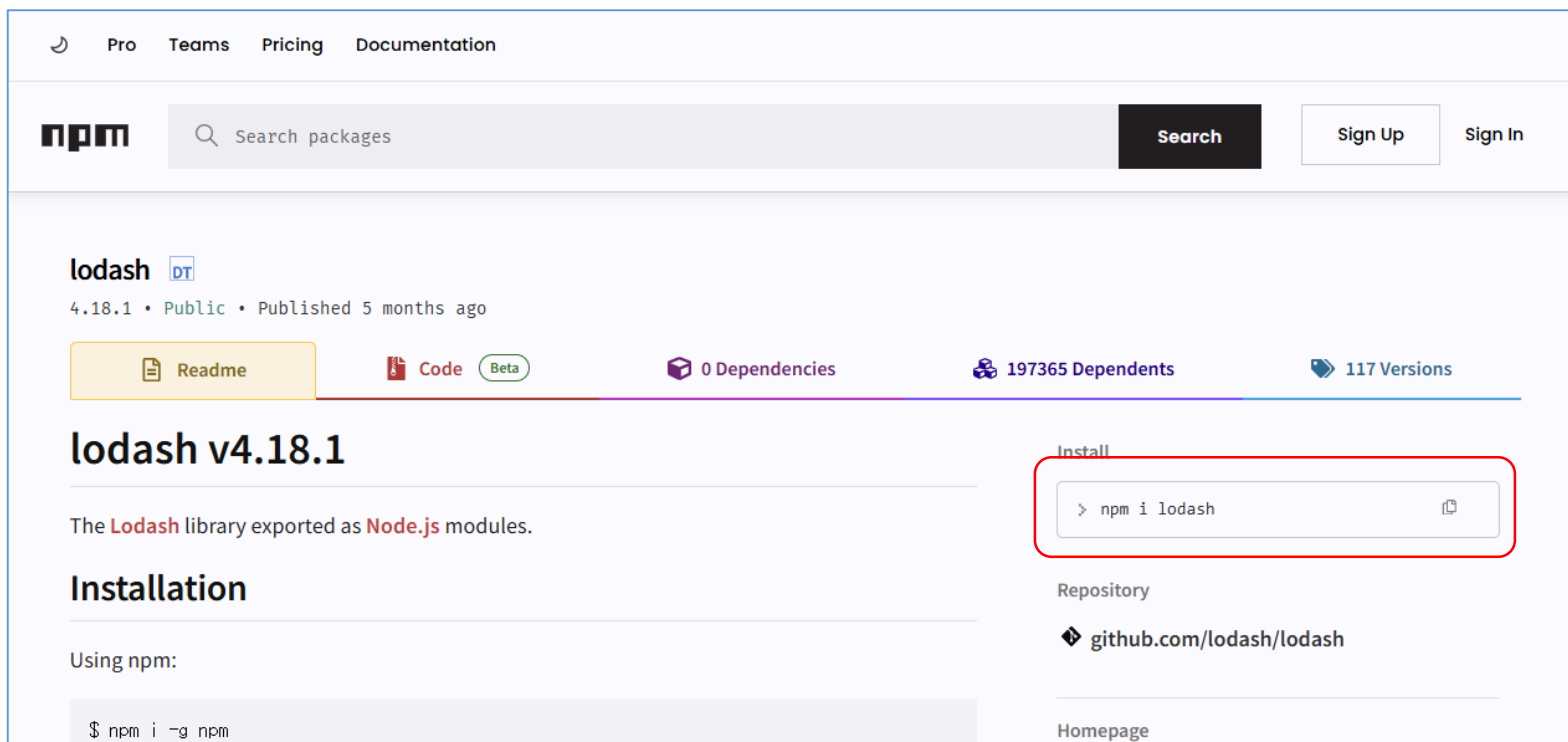
Node.js 라이브러리 사용하기

- Node.js 패키지에서는 외부 패키지를 설치해 사용할 수 있음
- 외부 패키지란 자신이 만든 Node.js 패키지를 서버에 올려 다른 사람도 사용할 수 있도록 배포한 파일
- 외부 패키지를 다른 말로 라이브러리하고 함
- 라이브러리는 프로그램을 개발할 때 공통으로 사용할 수 있는 기능들을 모아 모듈화한 것
- 라이브러리는 완전한 프로그램은 아니며 특정 기능만을 수행

Node.js 라이브러리 사용하기

● 라이브러리 설치하기

- 아래 url은, 패키지 관리 도구인 npm(Node Package Manager)을 직접 개발한 팀이 관리하는 웹 사이트
- <https://www.npmjs.com>
- 실습을 위해 'lodash' 라이브러리를 설치하고 사용해 보겠음



Node.js 라이브러리 사용하기

● 라이브러리 설치하기

(.venv)

```
$ npm i lodash # 혹은 npm install lodash
```

- 패키지 루트에 lodash 라이브러리를 저장하는 'node_modules' 폴더가 생성됨
- package.json에 lodash 라이브러리의 정보를 저장하는 dependencies 항목이 추가됨
 - Node.js 패키지에서 dependencies란 이 패키지를 실행하기 위해 필요한 추가 라이브러리라는 뜻
- 패키지 루트 아래에 package-lock.json이라는 이름의 파일이 생성됨
 - 설치된 라이브러리의 버전을 정확히 밝히기 위해 존재하는 파일
 - 이 파일을 별도로 생성하는 이유는 package.json의 dependencies에는 설치된 라이브러리의 정확한 버전이 아니라 버전의 범위(Version Range)만 있기 때문임
- 라이브러리 재설치
 - Node.js 패키지 공유 시 node_modules 폴더는 제외하고 공유
 - 패키지를 공유 받는 개발자는 다음의 명령으로 자동으로 설치됨
 - package.json과 package-lock.json에 있는 정보를 토대로 node_modules를 다시 만듦

(.venv)

```
$ npm install
```

Node.js 라이브러리 사용하기

- 라이브러리 사용하기

- index.js를 수정하고 실행하기

```
import lodash from "lodash";

const arr = [1, 1, 1, 2, 2, 1, 1, 4, 4, 3, 2];
const uniqueArr = lodash.uniqBy(arr);

console.log(uniqueArr);
```

- lodash 공식 문서
 - <https://lodash.com/docs>